

# Assignment 3: Model-Based Reinforcement Learning

Sertan Erdogan (s5152283) & Hossein Kelisa (s5152135)

Group Number: 59

## 1 Introduction

Model-free reinforcement learning methods such as Q-learning, learn entirely from direct environment interaction, requiring many real steps before converging to a good policy. Model-based reinforcement learning (MBRL) offers a more sample-efficient alternative: the agent simultaneously learns a model of the environment and uses that model to perform additional simulated updates, amplifying the learning signal obtained from each real step.

This report investigates two MBRL algorithms — Dyna and Prioritized Sweeping (PS) — applied to the Windy Gridworld: a  $10 \times 7$  grid in which an agent must navigate from start position  $(0, 3)$  to goal position  $(7, 3)$ . Reaching the goal yields  $+100$ ; every other step yields  $-1$ . A key environmental feature is wind: columns 3–5 and 8 push the agent upward by one cell per step, and columns 6–7 by two cells. The `wind_proportion` parameter controls how often the wind applies, creating either a stochastic environment (wind blows 90% of the time) or a deterministic one (wind always blows). All experiments use  $\epsilon = 0.1$ ,  $\alpha = 0.2$ ,  $\gamma = 1.0$ , and are averaged over 20 independent repetitions of 10 001 timesteps, with greedy evaluation every 250 steps.

## 2 Dyna

### 2.1 Methodology

Dyna [1] augments standard Q-learning with a planning phase driven by a learned environment model. At each timestep, the agent selects an action via an  $\epsilon$ -greedy policy, observes the transition  $(s, a, r, s')$ , and updates its model by incrementing transition counts and accumulating reward sums:

$$\hat{p}(s'|s, a) = \frac{n(s, a, s')}{\sum_{\tilde{s}} n(s, a, \tilde{s})}, \quad \hat{r}(s, a, s') = \frac{R_{\text{sum}}(s, a, s')}{n(s, a, s')} \quad (1)$$

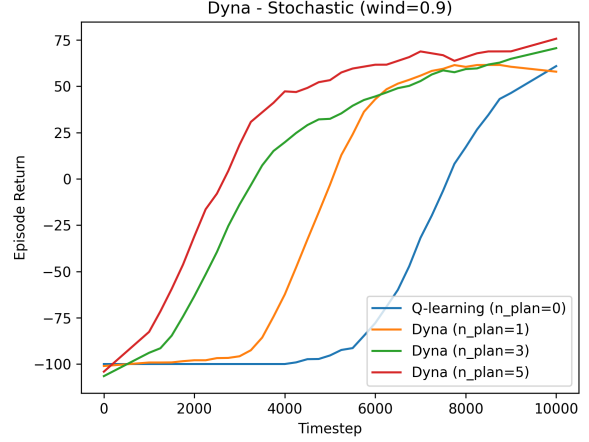
The real transition is then used for a standard Q-learning update [1, Eq. 6.8]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2)$$

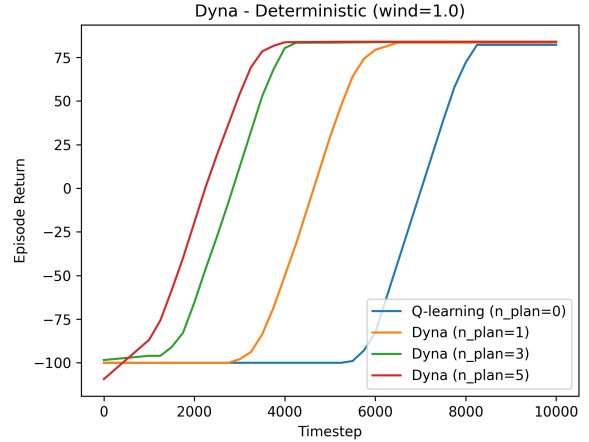
After this real update, the agent performs  $K$  planning steps. In each planning step, a previously visited  $(s, a)$  pair is sampled uniformly at random, a next state  $s'$  is drawn from  $\hat{p}(\cdot|s, a)$ , and Equation 2 is applied using the simulated transition. Each real step thus produces  $K+1$  Q-updates in total, directly improving sample efficiency by a factor proportional to  $K$ .

### 2.2 Results

Figure 1 shows Dyna learning curves in the stochastic environment ( $\text{wind} = 0.9$ ) for planning budgets  $K \in \{1, 3, 5\}$  alongside a Q-learning baseline ( $K = 0$ ). Figure 2 shows the same experiment in the deterministic environment ( $\text{wind} = 1.0$ ). All curves are averaged over 20 repetitions and lightly smoothed.



**Figure 1: Dyna learning curves in the stochastic environment ( $\text{wind} = 0.9$ ). Higher planning budgets lead to faster convergence, though the effect is softer than in the deterministic case due to model inaccuracy.**



**Figure 2: Dyna learning curves in the deterministic environment ( $\text{wind} = 1.0$ ). The effect of  $K$  is striking:  $K = 5$  converges roughly four times faster than Q-learning.**

### 2.3 Interpretation

In the deterministic environment, the benefit of planning is dramatic. With  $K = 5$ , the agent reaches near-optimal performance by approximately  $t = 2,000$ , whereas Q-learning ( $K = 0$ ) requires roughly  $t = 8,000$  steps. The curves are sharp and cleanly ordered by  $K$  because the model is perfectly accurate — since wind always

blows, every simulated transition faithfully reflects the real environment, making additional planning updates as informative as real ones.

When  $K$  is very small (e.g.  $K = 1$ ), the benefit over Q-learning is modest: a single additional simulated update per real step only marginally increases the effective learning rate, especially early in training when few state-action pairs have been visited and the pool of available planning transitions is limited.

In the stochastic environment, planning still accelerates learning but the advantage is reduced. Because the wind blows only 90% of the time, the learned model occasionally produces transitions that do not match reality, introducing noise into the planning updates. Nevertheless,  $K = 5$  converges considerably faster than Q-learning, demonstrating that even an imperfect model provides useful guidance. The curves also show more variance and a slower approach to the final asymptote, reflecting the harder exploration problem in a stochastic world. Notably, in the stochastic case, the final performance plateau is slightly lower ( $\approx 75$ ) than in the deterministic case ( $\approx 83$ ), since no policy can perfectly avoid wind-induced displacement when transitions are random.

### 3 Prioritized Sweeping

#### 3.1 Methodology

Prioritized Sweeping (PS) [1] replaces Dyna’s random planning step selection with a priority queue that focuses updates on the state-action pairs where the Q-estimate is most in error. After each real step, the absolute TD error is computed as the priority:

$$p = |r + \gamma \max_{a'} Q(s', a') - Q(s, a)| \quad (3)$$

If  $p > \theta$  (threshold  $\theta = 0.01$ ), the pair  $(s, a)$  is inserted into the priority queue. During each of the  $K$  planning steps, the highest-priority pair  $(\bar{s}, \bar{a})$  is popped, a Q-update is applied using the model, and then all predecessor pairs  $(\bar{s}, \bar{a}, s)$  satisfying  $n(\bar{s}, \bar{a}, s) > 0$  are evaluated. For each predecessor, a new priority is computed as:

$$p = |\hat{r}(\bar{s}, \bar{a}, s) + \gamma \max_{a'} Q(s, a') - Q(\bar{s}, \bar{a})| \quad (4)$$

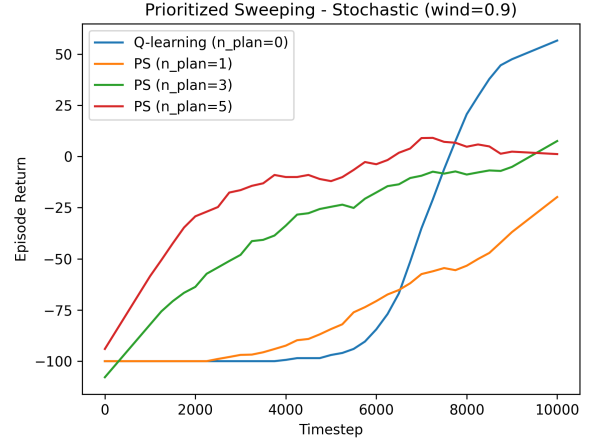
and those exceeding  $\theta$  are inserted into the queue. This backward propagation is the defining feature of PS: once the goal is reached and its value becomes large, that information is immediately propagated to predecessor states, spreading through the state space far faster than Dyna’s random replay.

#### 3.2 Results

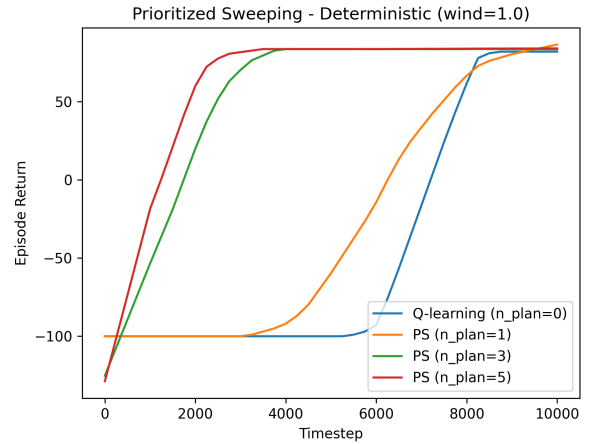
Figures 3 and 4 show PS learning curves for  $K \in \{1, 3, 5\}$  alongside the Q-learning baseline, in the stochastic and deterministic environments respectively.

#### 3.3 Interpretation

In the deterministic environment, PS is highly effective. With  $K = 5$ , the agent converges around  $t = 1,500$  — faster than Dyna at the same budget. The backward sweep efficiently propagates the +100 goal reward through all predecessor states, so value information flows from the goal to the start in far fewer steps than with random planning.

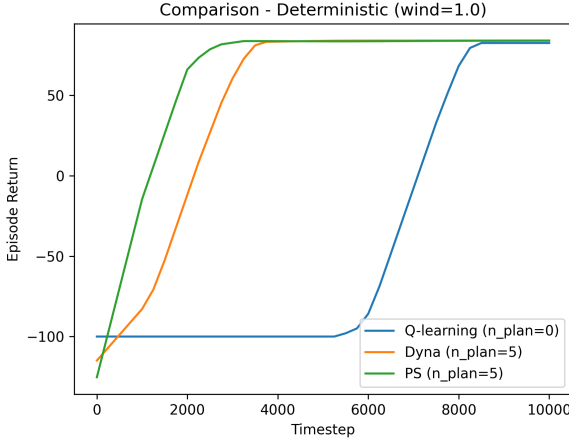


**Figure 3: Prioritized Sweeping in the stochastic environment (wind = 0.9). Remarkably, all PS variants perform worse than the Q-learning baseline throughout training.**



**Figure 4: Prioritized Sweeping in the deterministic environment (wind = 1.0). PS with  $K = 5$  converges fastest of all configurations, outperforming even Dyna at the same budget.**

In the stochastic environment, however, PS performs substantially worse than Q-learning. All PS variants — even  $K = 5$  — plateau far below the Q-learning baseline, reaching an episode return of only around 25 compared to Q-learning’s 60 by the end of training. This is a striking failure mode. PS’s backward sweep relies on the model’s priority estimates being meaningful: if the model inaccurately represents stochastic transitions, the priorities become misleading, and the backward propagation actively spreads incorrect value information through the state space. With more planning ( $K = 5$ ), this incorrect information is propagated more aggressively, making PS with higher  $K$  actually worse than PS with lower  $K$  in the stochastic setting.



**Figure 5: Algorithm comparison in the deterministic environment. PS converges fastest, followed by Dyna. Both comfortably outpace Q-learning.**

The failure is most pronounced at higher planning budgets: in Figure 3,  $K = 5$  performs worse than  $K = 3$ , which in turn performs worse than  $K = 1$  throughout most of training. This happens because the predecessor loop propagates high-priority but inaccurate updates deep into the state space before the model has accumulated enough samples to stabilize. Each backward sweep confidently updates many states based on unreliable transition estimates, and with more planning steps, this misinformation spreads further. Effectively, PS’s greatest strength in deterministic settings — its directed, confident backward propagation — becomes its greatest liability when the model it propagates through is noisy.

## 4 Comparison

### 4.1 Learning Curves

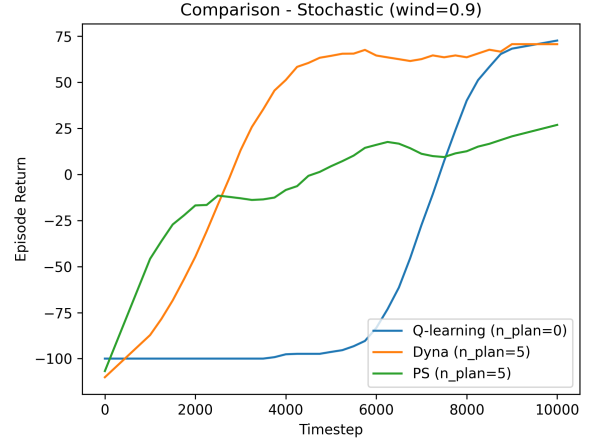
Figures 5 and 6 compare the best-performing Dyna ( $K = 5$ ), best-performing PS ( $K = 5$ ), and the Q-learning baseline in both environments.

In the deterministic environment (Figure 5), PS clearly learns fastest, followed by Dyna, with Q-learning a distant third. All three converge to the same final performance, confirming that differences are in sample efficiency rather than asymptotic quality. In the stochastic environment (Figure 6), the ranking reverses sharply: Dyna reaches approximately 65 return by  $t = 10,000$ , Q-learning converges more slowly but reaches a comparable level, and PS lags far behind both, reaching only  $\approx 25$ . This reversal highlights that PS’s advantage in the deterministic setting is precisely what makes it vulnerable in the stochastic setting.

### 4.2 Runtime Comparison

Table 1 reports the average wall-clock time per repetition (10 001 timesteps), measured over 5 timing repetitions.

Q-learning is the fastest because it performs only one update per time step. Dyna adds only modest overhead ( $\approx 30\%$  slower than Q-learning) because the random planning loop is computationally



**Figure 6: Algorithm comparison in the stochastic environment. The ranking reverses: Dyna outperforms Q-learning, while PS lags far behind both.**

**Table 1: Average runtime per repetition (seconds).**

| Algorithm              | Stochastic | Deterministic |
|------------------------|------------|---------------|
| Q-learning ( $K = 0$ ) | 6.00       | 5.99          |
| Dyna ( $K = 5$ )       | 7.87       | 7.16          |
| PS ( $K = 5$ )         | 17.94      | 14.80         |

cheap. PS is nearly three times slower than Q-learning, owing to the priority queue operations and the predecessor loop, which must scan all state-action pairs to find predecessors at every planning step — an  $O(|\mathcal{S}| \cdot |\mathcal{A}|)$  operation performed  $K$  times per timestep. Interestingly, PS’s runtime is relatively similar across the two environments (17.94s vs 14.80s), while Dyna shows a more noticeable gap (7.87s vs 7.16s). This is because in the stochastic environment the agent explores more broadly, visiting more state-action pairs and filling the planning pool more densely, which slightly increases the cost of both random sampling in Dyna and the predecessor search in PS. Both MBRL algorithms run slightly faster in the deterministic environment, likely because the policy converges earlier, leading to shorter episodes and fewer total environment steps. The much greater computational cost of PS is an important practical consideration. Despite its superior sample efficiency in deterministic settings, it may not be cost-effective when computation is also a limited resource.

These results highlight a fundamental tradeoff between sample efficiency and computational cost. In the deterministic setting, PS achieves near-optimal performance in roughly 1,500 timesteps while Q-learning requires approximately 8,000 — a roughly  $5\times$  improvement in sample efficiency at a cost of  $3\times$  more computation per repetition. Whether this tradeoff is worthwhile depends entirely on which resource is more scarce. In physical robotics or clinical settings where real interactions are expensive or dangerous, PS’s sample efficiency in deterministic environments makes it clearly preferable despite the computational overhead. In purely simulated

settings where real steps are cheap, Q-learning or Dyna may be more practical choices.

## 5 Reflection

### 5.1 Model-Based vs. Model-Free RL

The primary strength of MBRL is sample efficiency. By building and reusing a learned model, the agent extracts far more signal from each real interaction. In our experiments, Dyna with  $K = 5$  reached the same asymptotic performance as Q-learning in approximately one-fifth of the real timesteps. This advantage is especially valuable in settings where real interactions are expensive, such as physical robotics or clinical decision-making, where each real step may carry a high cost or risk.

The corresponding weakness is model dependence. A learned model is only an approximation, and planning with an inaccurate model can compound errors rather than correct them. Our PS results in the stochastic environment illustrate this failure mode clearly: backward propagation through a noisy model actively degraded performance below even the model-free baseline. Model-free methods like Q-learning are more robust in this respect, since they never introduce model bias — every update comes directly from real experience.

### 5.2 Dyna vs. Prioritized Sweeping

Both algorithms share the same underlying model but differ fundamentally in how planning steps are selected. Dyna’s random selection is computationally cheap and robust to model inaccuracy, since no single noisy simulated transition dominates the updates. PS’s priority-based selection is more targeted and therefore more efficient when the model is reliable, as demonstrated by its faster convergence in the deterministic environment. However, the same directedness makes PS brittle: confident backward propagation of inaccurate priorities actively harms performance in the stochastic case.

A further distinction is how each algorithm responds to increasing  $K$ . For Dyna, more planning is always beneficial regardless of environmental stochasticity, since random replay cannot systematically mislead the agent. For PS in the stochastic setting, however, increasing  $K$  actively hurts performance: Figure 3 shows  $K = 5$  consistently underperforming  $K = 1$  throughout most of training, because more planning steps give the noisy backward sweep more opportunity to corrupt Q-value estimates across the state space.

In this experiment, PS wins in the deterministic setting and Dyna wins in the stochastic setting, suggesting that the choice between them depends critically on how noisy the environment is. The two ideas are not mutually exclusive: one could combine them by using a priority queue for the majority of planning steps while occasionally inserting random planning steps. This hybrid would retain PS’s efficiency when the model is accurate, while providing a safety net of unbiased updates when priorities are misleading.

### 5.3 Exploration and Q-Value Initialization

All Q-values were initialized to zero, while every non-goal step yields  $-1$ . This creates a form of pessimistic initialization relative to the default step cost: every unvisited state has a Q-value of 0, which is higher than the actual return the agent will receive from those

states once it starts accumulating  $-1$  penalties. This encourages the agent to reach the goal quickly once it discovers it, since the  $+100$  goal reward makes those Q-values strongly positive and the agent learns to prefer paths leading toward the goal.

This stands in contrast to optimistic initialization, where Q-values are set to a large positive number to explicitly drive exploration. With optimistic initialization, the agent is initially drawn to every unvisited state because it expects high rewards — a systematic exploration bonus that disappears once a state is visited. The 0 initialization used here provides a much weaker exploration bonus, since the gap between the initial value and the true value is smaller.

The assignment further suggests examining the effect of setting the default step reward to 0. Without a per-step penalty, the agent has no intrinsic pressure to find the shortest path and early exploration becomes largely undirected — the agent wanders the grid with no cost signal distinguishing productive from unproductive actions. The only learning signal comes from accidentally reaching the goal, which in a 70-state grid with wind may take many episodes to occur by chance. In contrast, the  $-1$  step reward acts as a soft guidance signal: every step away from the goal is a small loss, so even before the agent has found the goal, Q-values begin to reflect the cost of distance. This is reminiscent of reward shaping [1], where auxiliary reward signals guide the agent toward productive regions of the state space without altering the optimal policy.

### 5.4 Limitations and Future Work

Several limitations of this study are worth acknowledging. First, we only tested  $K \in \{1, 3, 5\}$ ; larger planning budgets might reveal different tradeoffs, particularly for PS in the deterministic setting where even faster convergence may be achievable. Second, all experiments were conducted in a single small environment with only 70 states. In larger state spaces, the predecessor loop in PS becomes increasingly expensive — scaling as  $O(|S| \cdot |\mathcal{A}|)$  per planning step — which may make it impractical without approximations such as maintaining an explicit predecessor list.

Third, we kept  $\epsilon$ ,  $\alpha$ , and the priority threshold  $\theta$  fixed throughout training. Adaptive schedules — such as decaying  $\epsilon$  over time or adjusting  $\theta$  based on the variance of observed rewards — could improve performance in both environments, particularly for PS in the stochastic case where a higher threshold might filter out unreliable priorities more aggressively.

Future work could explore several promising directions. A hybrid Dyna-PS algorithm that combines priority-based and random planning steps could retain the efficiency of PS in deterministic settings while avoiding its failure mode in stochastic ones. Extending these methods to continuous state spaces using function approximation (e.g., neural network Q-functions) would be a natural next step toward real-world applicability. Finally, investigating the `wind_proportion` threshold at which PS begins to underperform Dyna could help practitioners make a principled choice between the two algorithms based on prior knowledge of environment stochasticity.

## References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.